

Lecture 17 - MDPs & Value / Policy Iteration

Reinforcement Learning

- MDPs (recap)
- Value function
- Value iteration / Policy iteration
- Learning P_{sa} / Putting it together

The way an MDP works

S_0

$$a_0 = \pi(S_0)$$

$$S_1 \sim P_{S_0 a_0}$$

a_1

$$S_2 \sim P_{S_1 a_1}$$

Total payoff = $R(S_0) + \gamma R(S_1) + \gamma^2 R(S_2) + \dots$

Policy $\pi: S \leftrightarrow A$

How to compute optimal policy →

Define: V^π , V^* , π^*

for a policy π , $V^\pi: S \mapsto \mathbb{R}$ is such that $V^\pi(s)$ is the expected total payoff for starting in state s and executing π .

$$V^\pi(s) = E \left[R(S_0) + \gamma R(S_1) + \dots \mid \pi, S_0 = s \right]$$

↓
Value function for policy π .

3	→	→	→	+1
2	↓		→	-1
1	→	→	↑	↑
	1	2	3	4

This is a bad policy.

~~V^π~~

0.52	0.73	0.77	+1
-0.9		-0.82	-1
-0.88	-0.87	-0.85	-1.00

V^π

These numbers are random.

Bellman's Equations.

~~$$V^{\pi}(s) = R(s_0) + \sum_{s'} P(s') \cdot V^{\pi}(s')$$~~

~~V^π~~

$$V^{\pi}(s) = R(s_0) + \sum_{s'} P(s') \cdot V^{\pi}(s')$$

~~$$V^{\pi}(s) = R(s_0)$$~~

$$s = s_0$$

↓
 immediate
 reward

In general,

$$V^{\pi}(s) = E \left[R(s_0) + \sum R(s_1) + \sum^2 R(s_2) \dots \mid s_0 = s \right]$$

$$V^\pi(s) = E \left[R(s_0) + \underbrace{f(R(s_1) + f(R(s_2)) \dots)}_{V^\pi(s_1)}, s_0 = s \right]$$

$$V^\pi(s) = E \left[R(s) + \underbrace{f}_{s_0} V^\pi(s') \right]$$

$s' \sim P_{s\pi(s)}$
 ↓
 after 1 time step

In state s , take action $a = \pi(s)$

$$s' \sim P_{sa}, \text{ where } s = \pi(s)$$

$$V^\pi(s) = R(s) + \sum_{s'} P_{s\pi(s)} V^\pi(s')$$

Given π , get a linear system of equations in terms of $V^\pi(s)$.

$$V^\pi(3,1) = R((3,1)) + \cancel{f} \left[(0.8 V^\pi(3,2) + 0.1 V^\pi(2,1) + 0.1 V^\pi(4,1)) \right]$$

$V^\pi(3,1)$, $V^\pi(3,2)$, $V^\pi(2,1)$ & $V^\pi(4,1)$ are unknown's. All these can be solved using system of linear equations with 11 equations & 11 unknowns.

$$\begin{bmatrix} V^{\pi}(1,4) \\ V^{\pi}(1,2) \\ \vdots \\ V^{\pi}(4,3) \end{bmatrix} \in \mathbb{R}^n$$

Using a linear algebra, we can solve 11 unknowns with 11 equations.

$V^* \rightarrow$ This is the optimal value function.

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

There is a different version of Bellman's equation $\rightarrow$

$$V^*(s) = R(s) + \max_a \underbrace{\sum_{s'} P(s'|sa) V^*(s')}_{\text{expected future reward.}}$$

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|sa) V^*(s')$$

↑
optimal policy

This gives us the strategy for finding the optimal policy for MDP.

$$V^*(s) = \underbrace{V^{\pi^*}}_{\text{value function for } \pi^*}(s) \geq V^{\pi}(s)$$

value function for π^*
 for every π, s

strategy $\rightarrow$ 1) Find V^*
 2) Use argmax equation to find π^*

Value iteration

Initialize $V(s) = 0$ for every s .

for every s , update

$$V(s) := R(s) + \max_a \sum_{s'} P_{sa}(s') V(s')$$

The algorithm would work, whether we use synchronous update or asynchronous update.

Synchronous update - all updates are done simultaneously

If we run MDP on the policy, V^* would be as belows $\rightarrow$

3	0.86	0.9	0.95	+1
2	0.81		0.69	-1
1	0.78	0.75	0.71	0.49
	1	2	3	4

$$W_i \sum_{s'} P_{sa}(s') V^*(s') =$$

$$0.8 \times 0.75 + 0.1 \times 0.69 + 0.1 \times 0.71 = 0.74$$

Policy Iteration $\rightarrow$ Initialize π randomly

Repeat: $\{$

set $V_i = V^\pi$ (i.e. solve Bellman's equation to get V^π)

set $\pi(s) : \arg \max_{sa} \sum_{s'} P_{sa}(s') V(s')$

$\}$

V^π - linear system of equations

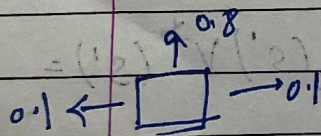
Value iteration converges to V^* .

In policy iteration, $\pi(s)$ might not change after some iterations.

Value iteration is used ~~for~~ more often because policy iteration can be computationally expensive to solve if there are millions of examples.

What if don't know P_{sa} ?

$P_{sa}(s') = \frac{\# \text{ times took action "a" in state } s \text{ and got to } s'}{\# \text{ times took action "a" in state } s}$



or $\frac{1}{151}$

if above is "0".

Putting it together:

Repeat

Take actions wrt π to get experience with in MDP

Update estimates of P_{sa}

Solve Bellman's eqⁿ: using value iteration to get V

update $\pi(s) = \arg \max_{s_i} \sum P_{sa}(s') V(s')$